
CO4 ACTIVITY

NAME: Jeya Harshini R

Reg no: 192424051

TITLE

Data Exploration and Analysis using Python (Pandas)

AIM

To perform data exploration and analysis by importing CSV datasets, exploring the dataset, applying table functions, and joining multiple datasets using the Pandas library in Google Colab.

SOFTWARE REQUIREMENTS

- Google Colab
 - Python 3
 - Pandas Library
 - CSV Files (sales.csv and customers.csv)
-

THEORY

Data Exploration is the process of examining a dataset to understand its structure, quality, and characteristics before analysis. It helps identify missing values, duplicate records, data types, and statistical summaries. After exploration, analytical functions are used to summarize the data, and multiple datasets are joined to obtain meaningful insights for decision-making.

PROCEDURE

1. Import the Pandas library.
2. Read the Sales and Customers CSV files.
3. Explore the dataset using Pandas functions.
4. Apply aggregate functions to summarize the data.
5. Join the Sales and Customers datasets using the CustomerID column.
6. Interpret the obtained results.

STEP 1 – IMPORTING DATA

Concept

Importing data is the process of loading external datasets into the analysis environment. CSV (Comma Separated Values) is one of the most commonly used formats for storing structured data. In this experiment, the datasets are imported into Pandas DataFrames using the `read_csv()` function.

Python Code

```
import pandas as pd
sales_df = pd.read_csv("/content/sales.csv")
customers_df = pd.read_csv("/content/customers.csv")

print("Datasets imported successfully!")
```

```
print("Sales Dataset")
display(sales_df)

print("Customers Dataset")
display(customers_df)
```

Output

```
... Datasets imported successfully!
```

Sales Dataset						
	SaleID	Product	Category	Price	Quantity	CustomerID
0	101	Laptop	Electronics	55000	2	1
1	102	Mouse	Electronics	800	5	2
2	103	Keyboard	Electronics	1200	3	3
3	104	Chair	Furniture	4500	1	1
4	105	Desk	Furniture	7000	2	2
5	106	Monitor	Electronics	15000	1	3
6	107	Printer	Electronics	9000	2	1
7	108	Notebook	Stationery	100	10	4
8	109	Pen	Stationery	20	50	5
9	110	Tablet	Electronics	25000	1	4

Customers Dataset			
	CustomerID	CustomerName	City
0	1	Ravi	Chennai
1	2	Priya	Coimbatore
2	3	Amit	Bangalore
3	4	Neha	Delhi
4	5	John	Mumbai

Observation

The Sales and Customers datasets were successfully imported from CSV files into Pandas DataFrames. The imported datasets are ready for exploration and analysis.

STEP 2 – EXPLORING DATA

Concept

Data exploration is the process of understanding the dataset before analysis. It includes checking the first few records, dataset structure, dimensions, data types, missing values, duplicate records, unique values, and summary statistics.

Operations Performed

- Display first five records
- Display dataset information

- Check dataset dimensions
- Identify data types
- Check missing values
- Check duplicate values
- Display unique categories
- Generate summary statistics

Python Code and Output

```
sales_df.head()
```

	SaleID	Product	Category	Price	Quantity	CustomerID
0	101	Laptop	Electronics	55000	2	1
1	102	Mouse	Electronics	800	5	2
2	103	Keyboard	Electronics	1200	3	3
3	104	Chair	Furniture	4500	1	1
4	105	Desk	Furniture	7000	2	2

```
sales_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   SaleID      10 non-null    int64
1   Product     10 non-null    object
2   Category    10 non-null    object
3   Price       10 non-null    int64
4   Quantity    10 non-null    int64
5   CustomerID  10 non-null    int64
dtypes: int64(4), object(2)
memory usage: 612.0+ bytes
```

```
sales_df.shape
```

```
(10, 6)
```

```
[ ] sales_df.dtypes
```

SaleID	int64
Product	object
Category	object
Price	int64
Quantity	int64
CustomerID	int64

dtype: object

```
[ ] sales_df.isnull().sum()
```

SaleID	0
Product	0
Category	0
Price	0
Quantity	0
CustomerID	0

dtype: int64

```
[ ] sales_df["Category"].unique()
```

array(['Electronics', 'Furniture', 'Stationery'], dtype=object)

```
[ ] sales_df.describe()
```

	SaleID	Price	Quantity	CustomerID
count	10.00000	10.000000	10.000000	10.000000
mean	105.50000	11762.000000	7.700000	2.600000
std	3.02765	17139.603133	15.114746	1.429841
min	101.00000	20.000000	1.000000	1.000000
25%	103.25000	900.000000	1.250000	1.250000
50%	105.50000	5750.000000	2.000000	2.500000
75%	107.75000	13500.000000	4.500000	3.750000
max	110.00000	55000.000000	50.000000	5.000000

Observation

The dataset contains 10 records and 6 columns. No missing or duplicate values were found. The summary statistics provide information such as count, mean, minimum, maximum, and standard deviation, which help in understanding the dataset before analysis.

STEP 3 – EXPLORING TABLE FUNCTIONS

Concept

Table functions summarize and analyze the data by performing mathematical and statistical operations. In Pandas, functions such as `count()`, `sum()`, `mean()`, `max()`, `min()`, `groupby()`, and `sort_values()` are used to generate meaningful insights from the dataset.

Functions Used

Function	Purpose
<code>count()</code>	Counts records
<code>sum()</code>	Calculates total
<code>mean()</code>	Calculates average
<code>max()</code>	Finds maximum value
<code>min()</code>	Finds minimum value
<code>groupby()</code>	Groups data
<code>sort_values()</code>	Sorts data

Python Code and Output

```
[ ] sales_df["Revenue"] = sales_df["Price"] * sales_df["Quantity"]
sales_df
```

	SaleID	Product	Category	Price	Quantity	CustomerID	Revenue
0	101	Laptop	Electronics	55000	2	1	110000
1	102	Mouse	Electronics	800	5	2	4000
2	103	Keyboard	Electronics	1200	3	3	3600
3	104	Chair	Furniture	4500	1	1	4500
4	105	Desk	Furniture	7000	2	2	14000
5	106	Monitor	Electronics	15000	1	3	15000
6	107	Printer	Electronics	9000	2	1	18000
7	108	Notebook	Stationery	100	10	4	1000
8	109	Pen	Stationery	20	50	5	1000
9	110	Tablet	Electronics	25000	1	4	25000

```
[ ] print("Count :", sales_df["Product"].count())
print("Total Revenue :", sales_df["Revenue"].sum())
print("Average Price :", sales_df["Price"].mean())
print("Maximum Price :", sales_df["Price"].max())
print("Minimum Price :", sales_df["Price"].min())
```

```
Count : 10
Total Revenue : 196100
Average Price : 11762.0
Maximum Price : 55000
Minimum Price : 20
```

```
[ ] sales_df.groupby("Category")["Revenue"].sum()
```

Revenue	
Category	
Electronics	175600
Furniture	18500
Stationery	2000

dtype: int64

```
sales_df.sort_values(by="Price", ascending=False)
```

	SaleID	Product	Category	Price	Quantity	CustomerID	Revenue
0	101	Laptop	Electronics	55000	2	1	110000
9	110	Tablet	Electronics	25000	1	4	25000
5	106	Monitor	Electronics	15000	1	3	15000
6	107	Printer	Electronics	9000	2	1	18000
4	105	Desk	Furniture	7000	2	2	14000
3	104	Chair	Furniture	4500	1	1	4500
2	103	Keyboard	Electronics	1200	3	3	3600
1	102	Mouse	Electronics	800	5	2	4000
7	108	Notebook	Stationery	100	10	4	1000
8	109	Pen	Stationery	20	50	5	1000

Observation

The aggregate functions successfully summarized the dataset. The total revenue, average product price, highest and lowest prices, category-wise revenue, and sorted data provided useful business insights.

STEP 4 – JOINING NUMEROUS DATASETS

Concept

Joining combines two or more datasets using a common column called a key. In this experiment, the Sales and Customers datasets are joined using the CustomerID column. This combines customer details with sales information, providing a complete dataset for analysis. The merge() function performs an Inner Join by default.

Why Join?

- Combines related information from multiple datasets.
- Provides a complete view of customer purchases.
- Makes reporting and analysis easier.
- Helps identify meaningful relationships between datasets.

Python Code and Output

```
[ ] merged_df = pd.merge(  
    sales_df,  
    customers_df,  
    on="CustomerID",  
    how="inner"  
)  
  
merged_df
```

	SaleID	Product	Category	Price	Quantity	CustomerID	Revenue	CustomerName	City
0	101	Laptop	Electronics	55000	2	1	110000	Ravi	Chennai
1	102	Mouse	Electronics	800	5	2	4000	Priya	Coimbatore
2	103	Keyboard	Electronics	1200	3	3	3600	Amit	Bangalore
3	104	Chair	Furniture	4500	1	1	4500	Ravi	Chennai
4	105	Desk	Furniture	7000	2	2	14000	Priya	Coimbatore
5	106	Monitor	Electronics	15000	1	3	15000	Amit	Bangalore
6	107	Printer	Electronics	9000	2	1	18000	Ravi	Chennai
7	108	Notebook	Stationery	100	10	4	1000	Neha	Delhi
8	109	Pen	Stationery	20	50	5	1000	John	Mumbai
9	110	Tablet	Electronics	25000	1	4	25000	Neha	Delhi

```
merged_df[
    "CustomerName",
    "City",
    "Product",
    "Category",
    "Price",
    "Quantity",
    "Revenue"
]
```

	CustomerName	City	Product	Category	Price	Quantity	Revenue
0	Ravi	Chennai	Laptop	Electronics	55000	2	110000
1	Priya	Coimbatore	Mouse	Electronics	800	5	4000
2	Amit	Bangalore	Keyboard	Electronics	1200	3	3600
3	Ravi	Chennai	Chair	Furniture	4500	1	4500
4	Priya	Coimbatore	Desk	Furniture	7000	2	14000
5	Amit	Bangalore	Monitor	Electronics	15000	1	15000
6	Ravi	Chennai	Printer	Electronics	9000	2	18000
7	Neha	Delhi	Notebook	Stationery	100	10	1000
8	John	Mumbai	Pen	Stationery	20	50	1000
9	Neha	Delhi	Tablet	Electronics	25000	1	25000

Observation

The Sales and Customers datasets were successfully merged using the CustomerID column. The resulting dataset combines customer information with product sales, making it easier to analyze customer purchasing patterns.

DATA EXPLORATION FLOW

Import Data

↓

Explore Data

↓

Analyze Data

↓

Join Datasets



Gain Insights

DATA QUALITY CHECK

- ✓ Data is accurate.
 - ✓ Data is complete.
 - ✓ No missing values were found.
 - ✓ No duplicate records were detected.
 - ✓ The dataset is suitable for analysis.
-

ADVANTAGES

- Helps understand the dataset before analysis.
 - Detects missing and duplicate values.
 - Generates statistical summaries quickly.
 - Simplifies data analysis using built-in functions.
 - Combines multiple datasets for better insights.
-

APPLICATIONS

- Sales Analysis
 - Customer Analysis
 - Business Intelligence
 - Marketing Analytics
 - Financial Reporting
 - Healthcare Data Analysis
-

RESULT

The Sales and Customers datasets were successfully imported into Google Colab using the Pandas library. Data exploration techniques were applied to understand the

structure and quality of the datasets. Aggregate functions were used to summarize the data, and the datasets were successfully joined using the `merge()` function to generate meaningful insights. Thus, the objectives of data exploration and analysis were achieved successfully.

192424051